Telecommunications Protocol: Complete Modelling and Verification Pipeline — Explained Easily

Your document explains how a telecommunications protocol is **modelled mathematically, tested for mistakes, and verified using Alloy**.

The main idea is simple:

When messages travel through several intermediate points, how can we be sure they reach the destination in the correct order, without being lost, skipped, or reordered?

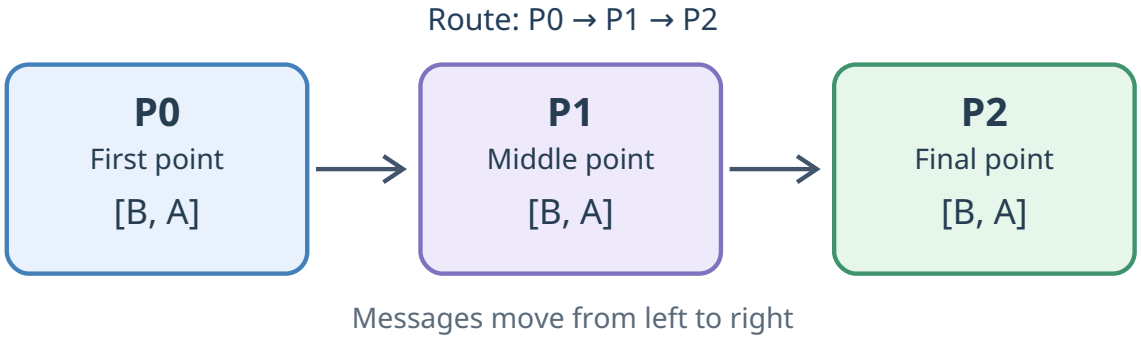
The project uses a mathematical model to describe the expected behaviour and then uses a verification tool called **Alloy Analyzer** to search for situations where that behaviour might be violated.

We will understand the basic concepts first, then the actual operations, and finally the testing and reporting pipeline.

1. Understand the problem using a real-world example

Imagine that you send two messages:

A telecommunications network with three signalling points



Conceptual diagram of the route. Each point can have its own pending queue; the identical lists shown are illustrative, not one simultaneous state.

Suppose you submit message A first, then message B.

The network should deliver:

A first, then B.

It must not deliver B first merely because B was submitted more recently.

Now imagine the messages have to travel through 20 intermediate points. At each point, some messages may be waiting while others have already moved to the next point.

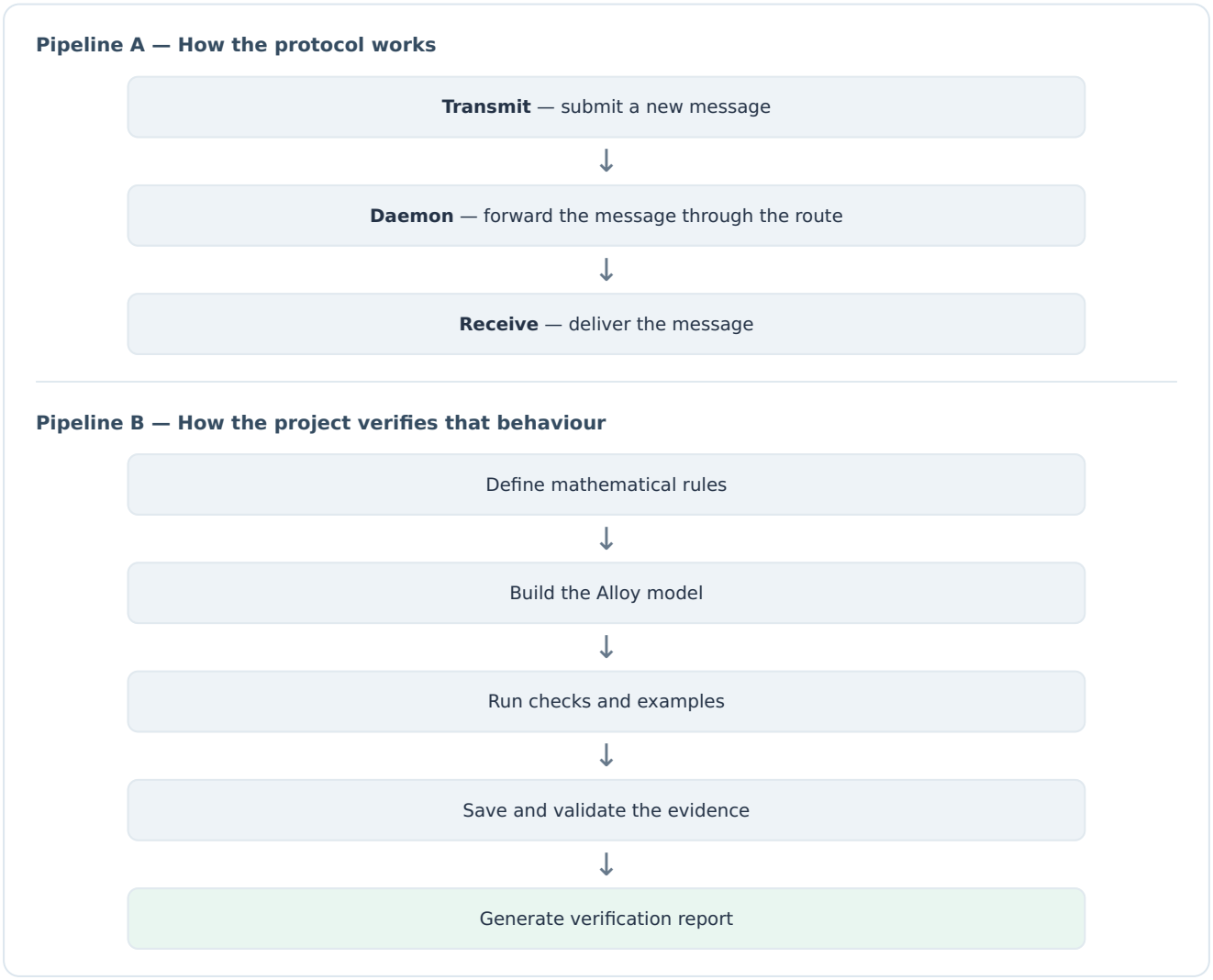
How do we know every intermediate operation preserves the correct order?

This is the problem your project addresses.

It does not simply run a few sample messages and assume the system is correct. It defines mathematical rules and asks Alloy to search systematically for violations within specified limits.

2. Two different pipelines are involved

Your document describes two connected processes.



The first pipeline describes **what the telecommunications system is allowed to do**.

The second pipeline describes **how the researcher checks whether those actions are correct**.

Understanding this distinction is essential.

3. External specification: What does the user see?

First, we ignore everything happening inside the network.

Imagine you are an outside observer. You can only see which messages were submitted and which messages were delivered.

The model calls these histories:

`input` — the complete history of submitted messages.

`output` — the complete history of delivered messages.

Important: Newest messages are written on the left

This is unusual, so understand it carefully.

Suppose messages arrive chronologically as A, then B, then C.

The model records:

```
input = [C, B, A]
```

C is on the left because it is the newest.

A is on the right because it is the oldest.

If A and B have already been delivered, the model records:

```
output = [B, A]
```

Notice that the actual delivery order was A then B, but the list displays B first.

The first mathematical safety rule

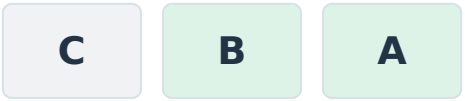
The document says:

The output must be a suffix of the input.

What is a suffix?

A suffix is the portion at the end, or right-hand side, of a sequence.

Example: input = [C, B, A]



✓ **Valid output**
[B, A]

✕ **Invalid output**
[C, A]

[B, A] is the right-hand portion of the input. [C, A] skips B, so it is not a suffix.

Mathematically, we write:

output is a suffix of input

This condition guarantees that the delivered history is an initial portion of the chronological submission history, in the same order.

But there is an important limitation:

This rule does not guarantee that all messages will eventually be delivered.

For example, `output = []` is valid even when the input contains several messages.

The rule is about safety, meaning nothing incorrect happens, not eventual delivery.

Based on the supplied document's external specification, sections 2 and 7.

4. Internal specification: What happens inside the network?

Now we open the network and examine its intermediate points.

Each signalling point is called an `SPC` in your model.

Each point maintains three sequences.

Symbol	Full meaning	Simple explanation
`rec[p]`	Received history	Everything that has arrived at point p
`ins[p]`	Pending queue	Messages still waiting at p
`sent[p]`	Sent history	Everything that has left p

Here, `p` simply identifies a particular signalling point.

For example, consider point P0.

Two messages, A and B, have arrived, but only A has been forwarded.

Its state is:

```
rec[P0]  = [B, A]
ins[P0]  = [B]
sent[P0] = [A]
```

The meaning is straightforward.

P0 has received A and B.

A has already left P0.

B is still waiting.

Why maintain three separate histories?

Because the system must be able to account for every message.

If the model only remembered the pending messages, it would lose the history of messages that had already passed through.

If it only remembered the arrival history, it would not know which messages were still waiting.

These three sequences let the model describe both the entire past and the current queue.

There is also a `route`, which records the ordered signalling points, such as `[P0, P1, P2]`.

The histories belong to each point's identity. If a new point is inserted into the route, existing points keep their histories.

5. The internal invariant: The most important rule

An **invariant** is a condition that must remain true in every valid state.

Your project has two especially important accounting rules.

Rule A: What arrived = what is waiting + what has left

$$\mathbf{rec[p] = ins[p] \mathbin{+\!\!+} sent[p]}$$

The symbol `++` means concatenate, or join two ordered sequences together.

It does not mean numerical addition.

For example:

$$\mathbf{[B] \mathbin{+\!\!+} [A] = [B, A]}$$

Therefore:

```
Received    [B, A]
             =
Pending     [B]
             +
Sent        [A]
```

This rule ensures that the complete arrival history is correctly divided into waiting and departed messages.

It also ensures the order is consistent, not just the number of messages.

Rule B: What leaves one point must arrive at the next

$$\mathbf{sent[P0] = rec[P1]}$$

Suppose P0 has forwarded A.

Then:

```
P0 sent history = [A]
P1 rec history  = [A]
```

Both histories agree.

If P0 says it sent A, but P1's arrival history is empty, the state would violate the model's rule.

These two rules combine to enforce consistency throughout the route.

> **Think of it as accounting for messages.**
>
> A bank must account for money received, money still held, and money transferred out. Your telecommunications model does something similar, but it also checks the exact order of message occurrences.

One subtle point: the model does not globally force every possible state to be valid. Instead, checks begin with a valid state and ask whether an operation can produce an invalid one. Otherwise, Alloy would be prevented from finding the mistakes the project is trying to detect.

6. The operations: How messages actually move

Now that the data and rules are clear, the operations become easier to understand.

Operation 1: `initial` — Create the starting state

Initially, all arrival, pending, and departure histories are empty.

For a two-point route:

```
P0: rec=[], ins=[], sent=[]
P1: rec=[], ins=[], sent=[]
```

The route itself must be nonempty, but this operation does not insist on a particular number of points.

At this moment, nothing has been submitted or delivered.

Operation 2: `transmit` — Submit a message

Suppose message A is submitted.

The first point records:

```
rec[P0] = [A]
ins[P0] = [A]
```

Its departure history remains empty because A has not yet left P0.

Now message B is submitted.

```
rec[P0] = [B, A]
ins[P0] = [B, A]
```

New entries go to the left.

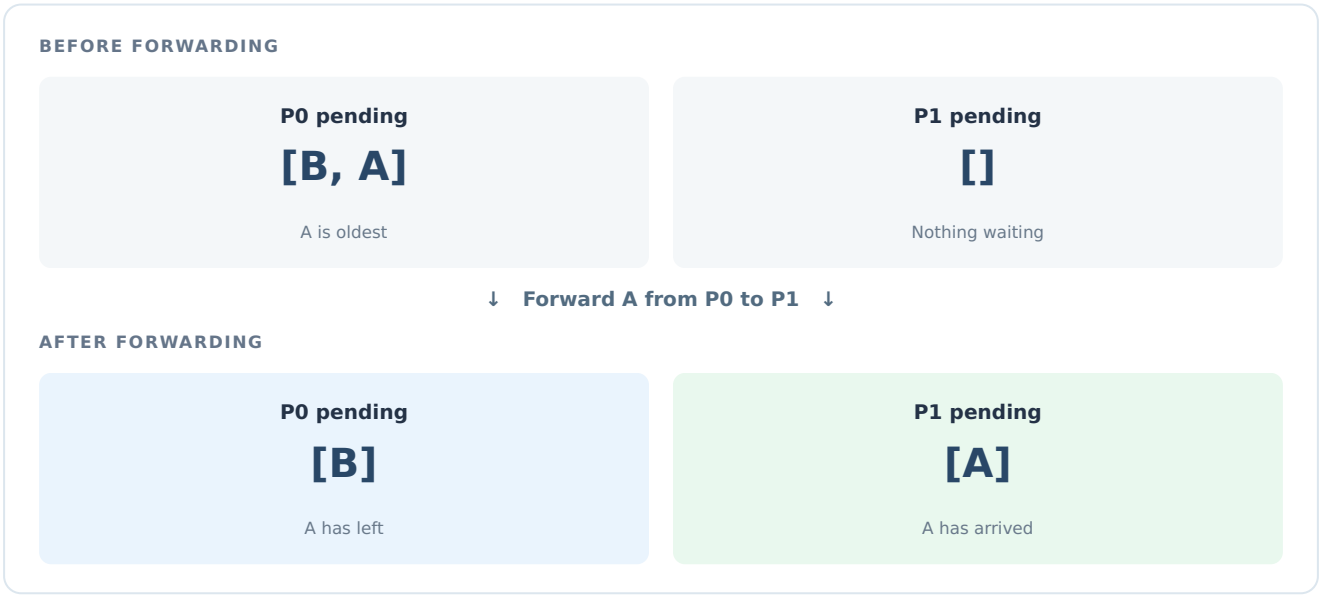
But the oldest pending message is A, on the right.

That matters in the next operation.

Operation 3: `daemon` — Forward a message

`daemon` is the operation that transfers a message from one point to its immediate successor.

Suppose:



Notice that B remains at P0, because A entered earlier.

Behind this simple movement, four things change simultaneously:

Field	Before	After
`ins[P0]`	[B, A]	[B]
`sent[P0]`	[]	[A]
`rec[P1]`	[]	[A]
`ins[P1]`	[]	[A]

Why four changes?

Because if we only moved A between the two pending queues, the arrival and departure histories would not agree with the operation.

The operation also explicitly preserves unrelated fields. These are called **frame conditions**.

In Alloy, a field does not automatically stay unchanged merely because an operation does not mention it.

Operation 4: `receive` — Deliver a message

A message can be delivered from the final point.

Suppose the last point contains:

```
ins[P1] = [B, A]
```

Successful `receive` removes A, the oldest message.

After delivery:

```
ins[P1]  = [B]
sent[P1] = [A]
```

And the external output becomes `[A]`.

The next successful receive delivers B:

```
ins[P1]  = []
sent[P1] = [B, A]
```

So A was delivered before B.

What is `receivePoll`?

The document distinguishes a successful delivery from a polling attempt.

A polling attempt can deliver a message, or it can leave everything unchanged.

That unchanged operation is called `stutter`.

This allows the model to represent a situation where the system is checked but nothing happens.

It is one reason the verified safety rules do not guarantee eventual delivery.

7. Inserting a new signalling point

This is an interesting part of the project.

Imagine the route is:

```
P0  →  P1
```

Now we want to add X between them.

```
P0  →  X  →  P1
```

The new route has three points.

But what happens if messages have already crossed from P0 to P1?

Suppose the recorded crossing history is `[A]`.

The new point X must be initialized as:

```
rec[X]  = [A]
ins[X]  = []
sent[X] = [A]
```

Why does X have an arrival and departure history when it has never physically processed A?

Because this is a mathematical reconstruction of the route's history.

The existing points already agree that A crossed their old boundary. X must inherit that history so the newly introduced boundaries remain consistent.

It is not claiming that A was physically transmitted through X in the past.

The new pending queue is empty, while the historical records preserve the accounting rules.

This operation is called `insertSection`.

8. Retrieve relation: Connecting the internal and external views

This is one of the most important ideas in formal verification.

We have two descriptions of the same system.

The external model sees only submitted and delivered messages.

The internal model sees every point, queue, and history.

We need a mathematical connection between them.

That connection is the **retrieve relation**.

Your project defines:

input[s] = rec[firstPoint[s]]

$$output[s] = sent[lastPoint[s]]$$

Here, `s` is a state snapshot.

In plain English:

External input is whatever the first point has received.

External output is whatever the final point has sent.

The model also constructs `inFlight`, which combines the pending queues along the route.

This produces a stronger equation:

$$input = inFlight ++ output$$

Example:

```
Input       = [C, B, A]

In flight   = [C]

Output      = [B, A]
```

Combining the waiting messages with the delivered history gives `[C,B,A]`, exactly the input.

Why is this useful?

Because if the internal operations preserve this relationship, then the internal implementation maintains the promised external ordering.

What does refinement mean here?

Refinement means that the detailed implementation behaves consistently with the simpler external specification.

For example, if the internal model forwards a message between two points, the external histories should not change. The message was neither newly submitted nor finally delivered.

Likewise, inserting an intermediate point should not change the externally observable input, output, or overall in-flight sequence.

The project checks these observable relationships within Alloy's finite bounds. It does not claim a complete unbounded refinement proof.

9. A complete example with two messages

Now combine everything.

Your document includes an actual saved Alloy example, not just an imagined scenario.

It has two route points and seven states connected by six operations.

Step	Operation	P0 pending	P1 pending	External output
0	Initial	[]	[]	[]
1	Transmit A	[A]	[]	[]
2	Transmit B	[B,A]	[]	[]
3	Forward A	[B]	[A]	[]
4	Forward B	[]	[B,A]	[]
5	Receive A	[]	[B]	[A]
6	Receive B	[]	[]	[B,A]

The external input starts empty, becomes `[A]` at step 1, then `[B,A]` at step 2, and remains `[B,A]` through step 6.

At the end, both messages have been delivered, with A before B.

That is a successful example.

But one example cannot prove that all possible executions are correct.

This is why the project also uses Alloy assertions and counterexample searches.

10. What does Alloy actually do?

Alloy is a tool for exploring mathematical models.

You describe objects, relationships, rules, and operations. Alloy searches for assignments that satisfy those definitions.

There are two particularly important command types: `run` and `check`.

Command	What it asks	Meaning of SAT
`run`	Can this scenario happen?	Yes, an example exists
`check`	Can this assertion be violated?	Yes, a counterexample exists

For a `check`, **UNSAT is usually the desired result** because it means no counterexample was found within the specified bounds.

This is the opposite of how people sometimes interpret SAT as an unconditional success.

Example: Testing a deliberate bug

Your project intentionally defines a faulty delivery operation:

`receiveNewestFirst_BUG`

Normally, the system should deliver the oldest pending message, on the right.

The buggy operation incorrectly delivers the newest pending message, on the left.

Suppose:

```
rec  = [B, A]
ins  = [B, A]
sent = []
```

If the buggy operation delivers B first:

```
rec  = [B, A]
ins  = [A]
sent = [B]
```

Now test the invariant:

$$\mathbf{rec = ins \mathbin{++} sent}$$

The right-hand side becomes:

$$\mathbf{[A] \mathbin{++} [B] = [A, B]}$$

But `rec` is `[B,A]`.

Therefore:

$$\mathbf{[B, A] \neq [A, B]}$$

The invariant is broken.

This explains why delivering the wrong queue element is a genuine error, not merely a different design choice.

Alloy found an actual saved counterexample to the intentional bug. That saved example is more complicated than this teaching illustration because it includes repeated message values.

The model keeps this buggy operation isolated from the correct operations and successful demonstration.

11. What did the verification results show?

According to the execution record described in your document, the selected completed run used Alloy 6.2.0 with the glucose solver.

18 recorded outcomes matched expectations

10	7	1
Safety checks	**Positive runs**	**Intentional bug**
No counterexample	Examples found	Counterexample found

The ten correct assertions returned UNSAT.

The seven requested positive scenarios returned SAT, establishing that those scenarios are possible.

The intentional bug check returned SAT, demonstrating that the flawed operation can violate the property.

These are the expected results for the selected bounded run.

But what exactly was the search limit?

The main correctness checks used bounds including:

Item	Recorded bound
Distinct message values	At most 4
Signalling points	At most 4
Sequence length	At most 4
State snapshots	1 or 2
Integer size	Signed 5-bit
Complete delivery example	7 snapshots, 2 messages, 2 points

These finite limits are important.

If Alloy reports UNSAT for a check, it means it could not find a counterexample **inside that specific search space**.

It does not mean the protocol has been proven correct for any imaginable number of messages, points, or operations.

One additional subtlety: a message value can occur more than once. For instance, `[A,A]` represents two message occurrences with the same value. The model uses sequence positions to distinguish those occurrences.

12. Why save evidence after running Alloy?

Your project does more than show a PASS/FAIL result.

It also attempts to make the results reproducible and auditable.

Imagine someone runs verification on Monday and modifies the model on Tuesday.

If the report does not record which version was tested, a result from Monday could accidentally be presented as proof about Tuesday's model.

Your pipeline addresses that risk.



What is SHA-256?

SHA-256 is a hashing algorithm that produces a fingerprint of file contents.

When the file contents change, the fingerprint will almost certainly change too.

The project records the hash of the saved model source so it can tell whether the evidence is being associated with the same source.

A matching hash supports source identity; it does not prove that the model itself is correct.

What does the validation script check?

The document describes comparisons between the frozen source, Analyzer output, and execution receipts.

These include whether the expected commands ran, whether the search bounds match, and whether SAT results have saved XML evidence.

The export script also validates the structure and meaning of the demonstration examples. For instance, it checks that the delivery scenario actually delivers A before B, and that the deliberate bug example truly violates the intended accounting rule.

The audit separately records **14 passing regression tests** for these reporting utilities.

These 14 tests are not another 14 protocol correctness proofs. They test whether the evidence-processing system can detect bad or incomplete records.

13. What has actually been proved, and what has not?

This is particularly important if you are writing a project report or research paper.

Supported by the supplied results	Not established
Valid example executions exist	Every execution eventually delivers
Ten intended assertions had no counterexample within bounds	Correctness for unlimited sizes
The intentional faulty operation has a counterexample	Full real-world network correctness
Observed abstract and concrete behaviours are compatible within bounds	Complete unbounded data refinement
Execution evidence is checked for consistency	Timing or real-time guarantees
Correct ordering is preserved in tested scenarios	Fault recovery, retransmission, or full SS7 conformance

Your document is careful about this distinction.

A **safety property** asks whether something bad can happen, such as delivering messages in the wrong order.

A **liveness property** asks whether something good will eventually happen, such as every pending message eventually being delivered.

Your model's polling and idle operations allow a state to remain unchanged, so safety alone cannot establish liveness.

The project also does not establish complete deadlock freedom, fairness, timing guarantees, or real-world telecommunications compliance.

14. How all the concepts fit together

Now the entire process can be understood as four layers.

1 **Layer 1 — External promise**
The outside world should see messages delivered in submission order. We describe that using `input`, `output`, and the suffix rule.

2 **Layer 2 — Internal mechanism**
Messages move through signalling points using `transmit`, `daemon`, `receive`, and `insertSection`. The internal invariant enforces local accounting and boundary consistency.

3 **Layer 3 — Mathematical verification**
The retrieve relation connects the internal histories with externally visible behaviour. Alloy searches for examples and counterexamples within finite bounds.

4 **Layer 4 — Reproducible evidence**
The Python runner saves the model used, executes checks, validates results, and produces reports so the claims can be traced back to the actual run.

15. Final understanding

The main contribution of the project is not inventing a new telecommunications routing algorithm.

It is taking a specified telecommunications behaviour, representing it precisely in Alloy, and constructing a repeatable process for checking that its internal operations preserve the promised external ordering rules.

The most important mathematical idea is:

$$\text{Input} = \text{Pending messages} + \text{Delivered messages}$$

where `+` means ordered sequence concatenation.

The most important verification idea is:

Start from a valid state, apply an operation, and search for a resulting state that breaks the invariant.

And the most important scientific limitation is:

No counterexample found within finite bounds is strong bounded evidence, not a universal proof for all possible network sizes and executions.

Once you understand these three ideas, the roles of the queues, operations, retrieve relation, Alloy checks, and verification reports all become much clearer.